

**AI335L Deep Learning Lab**  
Experiment No. 10 | Phase 3 of 6

---

**Architecture Report**  
LowLight Image Enhancement and Object Detection  
*ZeroDCE + YOLOv8 Pipeline*

|                |                                                                                                                                                           |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Course         | AI335L Deep Learning                                                                                                                                      |
| Phase          | Phase 3 of 6 — Architecture Report                                                                                                                        |
| Team Members   | Nimra Waseem S2024AI002<br>Umair Naveed S2024AI007<br>Dayan Amjad S2024AI007                                                                              |
| Repository     | <a href="https://github.com/Nimra-waseem-aftab/image_enhancement_and_detection">https://github.com/Nimra-waseem-aftab/image_enhancement_and_detection</a> |
| Commit Hash    | bc5b0531e7b720af90b97b220cabd291960bb465                                                                                                                  |
| Submission Tag | Phase3submission                                                                                                                                          |
| Date           | May 11, 2026                                                                                                                                              |

**1. Architecture Justification**

---

**1.1 Track Selection**

This project follows the CNN track. The problem domain is unsupervised lowlight image enhancement, where the input is a degraded RGB image and the output is a spatiallyenhanced version of the same image. CNNs are the appropriate choice because the problem is inherently spatial: the model must reason about local pixel neighborhoods, edge structure, texture, and illumination gradients. These are precisely the inductive biases that convolutional layers encode through learned spatiallyshared filters. An MLP applied pixelbypixel cannot exploit spatial context at all, as demonstrated by the MLP baseline implemented in the code. An RNN or Transformer would add unnecessary sequenceorder or attention

machinery to a problem that has no strict temporal or longrange nonlocal dependency at the architectural level. A CNN with skip connections is the direct match.

**1.2 Architecture Variant**

The primary architecture is ZeroDCE (ZeroReference Deep Curve Estimation), implemented as DCENet. This is a 7layer fullyconvolutional UNetstyle network that predicts 24channel pixelwise curve parameter maps. The 24 channels represent 8 iterative enhancement rounds, each operating on 3 RGB channels. An enhanced variant, DCENetPro, adds BatchNorm2d, MaxPool2d downsampling with bilinear upsample decoding, and Dropout2d regularisation.

The choice of curve estimation rather than direct pixel prediction is a deliberate architectural decision: instead of predicting an enhanced image directly, DCENet predicts parameters  $r$  for the function  $x_{\text{new}} = x + r * (x^2 - x)$ , applied 8 times. This parameterised tonemapping approach is more stable than direct pixel regression and produces smooth, monotonic intensity adjustments.

**1.3 Pretrained Backbone**

DCENet is trained from scratch. No pretrained backbone is used. This is appropriate because ZeroDCE is an unsupervised model with a unique loss formulation that does not align with any supervised pretraining objective. The architecture is intentionally lightweight (approximately 79,000 parameters) to prevent overfitting on the LOLv2 training set, which contains roughly 800 image pairs. Using a large pretrained backbone would introduce capacity mismatch and would not leverage pretrained representations meaningfully for this task.

YOLOv8, used in the downstream detection phase, is loaded with pretrained weights from COCO and used in zeroshot inference mode. No finetuning is applied to YOLOv8; its weights are frozen throughout the pipeline.

**1.4 Input Shape, Output Shape, and Parameter Count**

| Property                      | Value                 |
|-------------------------------|-----------------------|
| Input shape (DCENet)          | (B, 3, 256, 256)      |
| Output shape (DCENet)         | (B, 24, 256, 256)     |
| DCENet total parameters       | approximately 79,000  |
| DCENetPro total parameters    | approximately 286,000 |
| MLP Baseline total parameters | approximately 9,600   |
| Image size (training)         | 256 x 256 pixels      |
| Batch size                    | 16                    |

|                                  |                         |
|----------------------------------|-------------------------|
| Number of enhancement iterations | 8                       |
| Curve channels per iteration     | 3 (one per RGB channel) |

1.5 Inductive Biases

The CNN architecture brings three inductive biases that are wellmatched to this task. First, translation equivariance: the same enhancement operation should be applied regardless of where in the image a dark region appears. Second, local receptive fields: illumination in one patch is most strongly related to neighboring patches, not distant ones. Third, parameter sharing across spatial positions: the filters learned for one region generalise to all regions.

The UNetstyle skip connections in DCENet additionally allow the decoder to access fine spatial detail from the encoder, preventing the loss of highfrequency structure (edges, textures) during enhancement. Tanh activation in the final layer allows curve parameters  $r$  to be positive (brighten) or negative (darken), which is physically meaningful for tone mapping.

2. Model Architecture

2.1 DCENet Architecture Diagram

The diagram below shows the layerbylayer structure of DCENet. The encoder uses 4 convolutional blocks (conv1 through conv4). The decoder reconstructs spatial resolution using 3 convolutional blocks (conv5 through conv7), each receiving concatenated skip connections from the corresponding encoder layer. The final layer reduces channels to 24 and applies Tanh activation.

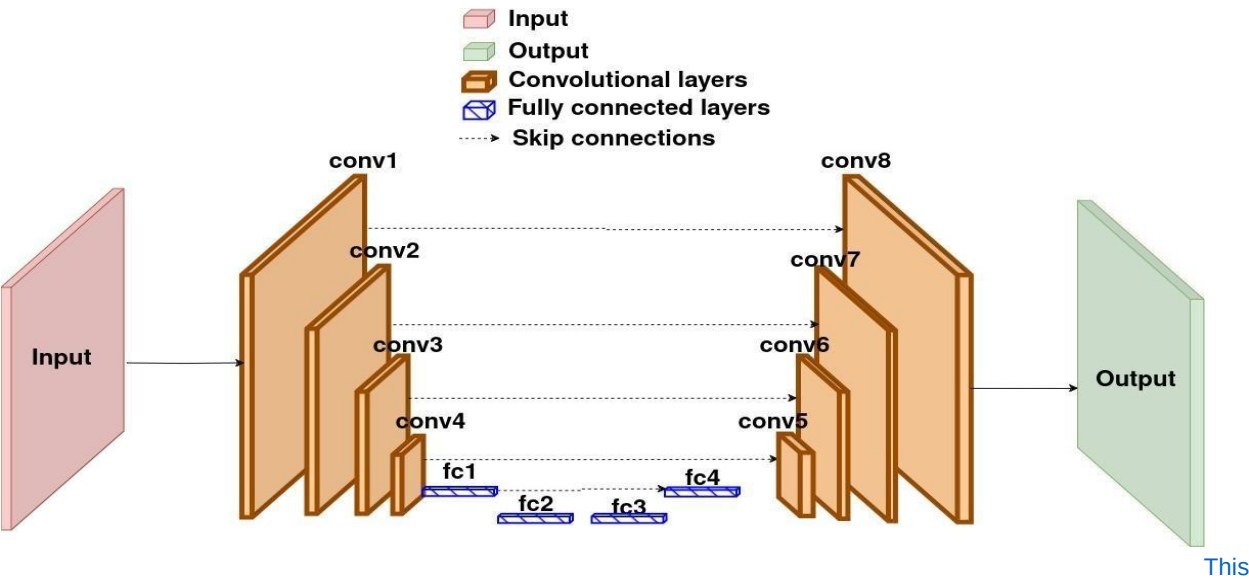
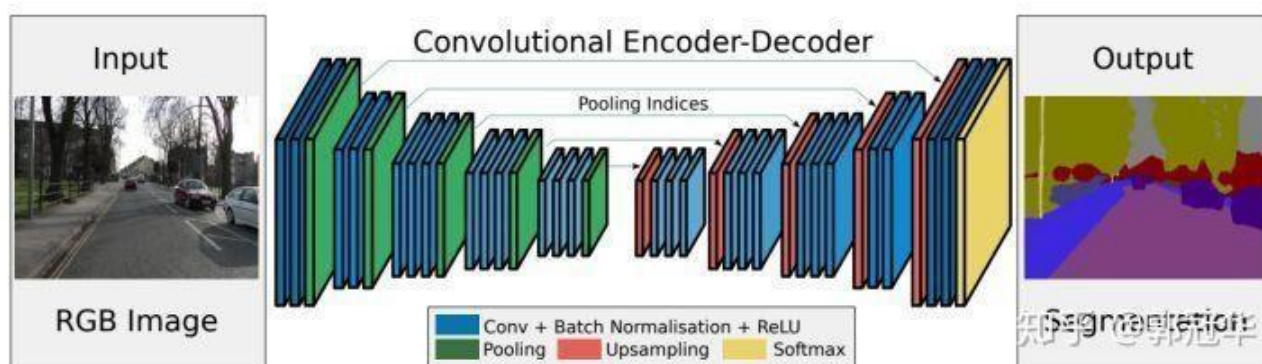


Photo by Unknown Author is licensed under CC BY

The DCENetPro variant augments this design with MaxPool2d downsampling in the encoder (256 to 128, then 128 to 64), BatchNorm2d after each convolution, Dropout2d before the final output, and bilinear upsample decoding. The diagram below shows DCENetPro.



[This Photo](#) by Unknown Author is licensed under [CC BY-SA](#)

## 2.2 LayerbyLayer Table: DCENet (Original)

| Layer | Type                       | Input Channels | Output Channels | Kernel    | Output Shape      | Parameters |
|-------|----------------------------|----------------|-----------------|-----------|-------------------|------------|
| conv1 | Conv2d + ReLU              | 3              | 32              | 3x3 pad 1 | (B, 32, 256, 256) | 896        |
| conv2 | Conv2d + ReLU              | 32             | 32              | 3x3 pad 1 | (B, 32, 256, 256) | 9,248      |
| conv3 | Conv2d + ReLU              | 32             | 32              | 3x3 pad 1 | (B, 32, 256, 256) | 9,248      |
| conv4 | Conv2d + ReLU              | 32             | 32              | 3x3 pad 1 | (B, 32, 256, 256) | 9,248      |
| conv5 | Cat(x4,x3) + Conv2d + ReLU | 64             | 32              | 3x3 pad 1 | (B, 32, 256, 256) | 18,464     |
| conv6 | Cat(x5,x2) + Conv2d + ReLU | 64             | 32              | 3x3 pad 1 | (B, 32, 256, 256) | 18,464     |
| conv7 | Cat(x6,x1) + Conv2d + ReLU | 64             | 32              | 3x3 pad 1 | (B, 32, 256, 256) | 18,464     |
| final | Conv2d + Tanh              | 32             | 24              | 3x3 pad 1 | (B, 24, 256, 256) | 6,936      |

Total parameters (DCENet original): approximately 79,000 trainable parameters.

### 2.3 LayerbyLayer Table: DCENetPro

| Layer      | Type                                      | In Ch | Out Ch | Output Shape      | Parameters |
|------------|-------------------------------------------|-------|--------|-------------------|------------|
| enc1       | Conv2d + BN + ReLU                        | 3     | 32     | (B, 32, 256, 256) | ~1,000     |
| pool1      | MaxPool2d 2x2                             | 32    | 32     | (B, 32, 128, 128) | 0          |
| enc2       | Conv2d + BN + ReLU                        | 32    | 64     | (B, 64, 128, 128) | ~18,600    |
| pool2      | MaxPool2d 2x2                             | 64    | 64     | (B, 64, 64, 64)   | 0          |
| bottleneck | Conv2d + BN + ReLU                        | 64    | 64     | (B, 64, 64, 64)   | ~37,000    |
| up1 + dec1 | Upsample + Cat(enc2) + Conv2d + BN + ReLU | 128   | 32     | (B, 32, 128, 128) | ~37,000    |
| up2 + dec2 | Upsample + Cat(enc1) + Conv2d + BN + ReLU | 64    | 32     | (B, 32, 256, 256) | ~18,600    |
| dropout    | Dropout2d (p=0.1)                         | 32    | 32     | (B, 32, 256, 256) | 0          |
| final      | Conv2d + Tanh                             | 32    | 24     | (B, 24, 256, 256) | ~7,000     |

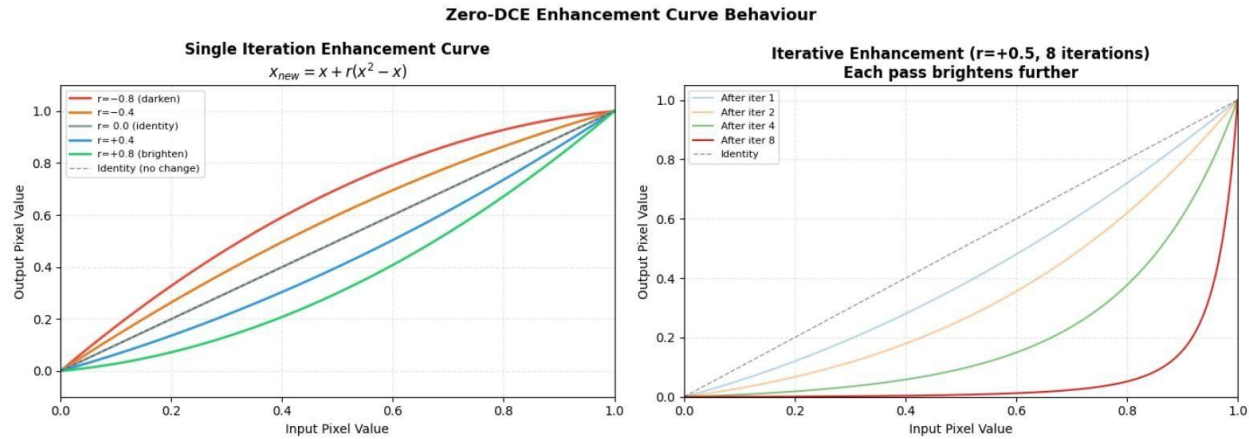
Total parameters (DCENetPro): approximately 286,000 trainable parameters. The increase relative to DCENet is due to the wider bottleneck (64 channels) and the BatchNorm parameter additions.

### 2.4 MLP Baseline Architecture

The MLP baseline treats each pixel independently as a 3dimensional input (R, G, B) and applies the following sequential layers: Linear(3 to 64), BatchNorm1d(64), ReLU, Dropout(0.1), Linear(64 to 64), BatchNorm1d(64), ReLU, Dropout(0.1), Linear(64 to 3), Sigmoid. The forward pass reshapes the input image from (B, 3, H, W) to (B\*H\*W, 3), applies the network, and reshapes back. Weights are initialised using Kaiming uniform initialisation. Total parameters: approximately 9,600. This model serves as the lowerbound reference point.

### 2.5 Enhancement Curve

The curve function applied after DCENet is:  $x_{new} = x + r * (x^2 - x)$ , where  $x$  is the input pixel tensor and  $r$  is the predicted curve parameter map. This formula is applied 8 times iteratively, using 3 channels of the 24channel output per iteration. When  $r$  is positive, the curve is concaveup (brightens midtones and highlights). When  $r$  is negative, it darkens. The iterative application allows finegrained, stable tone adjustment without direct pixel regression.



## 3. Training Setup

### 3.1 Dataset: LOLv2

The model is trained on the LOLv2 (LowLight version 2) dataset, which contains two subsets. The Synthetic subset comprises approximately 689 paired lowlight and normallight images generated by applying synthetic noise and degradation to normal images. The RealCaptured subset comprises approximately 100 pairs captured under real dark conditions. Both subsets are combined for training, giving approximately 789 total lowlight training images.

For ZeroDCE training, only the lowlight images are used. Normallight images are reserved exclusively for PSNR and SSIM evaluation. The dataset is split by shuffling with a fixed seed (42) before partitioning: 90 percent for training and 10 percent for validation.

| Split                      | Count                    | Source                                          |
|----------------------------|--------------------------|-------------------------------------------------|
| Training (lowlight only)   | ~710                     | LOLv2 Synthetic + Real Train Low                |
| Validation (lowlight only) | ~79                      | LOLv2 Synthetic + Real Train Low (10% held out) |
| Test (for PSNR/SSIM)       | Synthetic: 100, Real: 20 | LOLv2 Test Low                                  |
| Reference (PSNR/SSIM eval) | Matches test count       | LOLv2 Test Normal                               |

### 3.2 Preprocessing and Data Augmentation

Input images are resized to 256 by 256 pixels. Pixel values are normalised to the range [0, 1] using torchvision ToTensor(). No perimage mean subtraction or ImageNet normalisation is applied, as ZeroDCE is unsupervised and the curve formula operates on raw [0, 1] values.

Training augmentations are applied only to the training split to prevent label leakage into validation. The augmentation pipeline consists of: RandomHorizontalFlip (p=0.5), RandomRotation (up to 15 degrees), ColorJitter (brightness 0.3, contrast 0.3), and RandomResizedCrop (scale 0.8 to 1.0, ratio 0.9 to 1.1). Validation and test images receive only deterministic Resize and ToTensor.

### 3.3 Hyperparameters

| Hyperparameter            | Value             | Notes                                                     |
|---------------------------|-------------------|-----------------------------------------------------------|
| Image size                | 256 x 256         | Balances detail and VRAM; fullyconvolutional at inference |
| Batch size                | 16                | Stable gradient estimates on ~800image dataset            |
| Epochs                    | 200               | With early stopping (patience=20)                         |
| Learning rate             | 1e4               | Adam optimizer standard starting point for image tasks    |
| Weight decay              | 1e4               | L2 regularisation coefficient                             |
| Gradient clip norm        | 1.0               | Applied via clip_grad_norm_ before every optimizer step   |
| Optimizer                 | Adam              | Adaptive learning rates per parameter                     |
| Scheduler                 | CosineAnnealingLR | T_max=200, eta_min=1e6 (1% of initial LR)                 |
| Loss weight: smoothness   | 200               | Dominant term; enforces spatial smoothness of curve maps  |
| Loss weight: spatial      | 1                 | Local contrast preservation                               |
| Loss weight: color        | 5                 | RGB channel balance penalty                               |
| Loss weight: exposure     | 10                | Targets mean patch brightness of 0.6                      |
| Validation split          | 10%               | Fixed seed=42 shuffle before split                        |
| Random seeds (comparison) | 42, 123, 7        | Three seeds used for mean +/- std reporting               |
| Mixed precision (AMP)     | Enabled if CUDA   | torch.cuda.amp GradScaler                                 |
| Early stopping patience   | 20 epochs         | Halts training if val loss does not improve               |

### 3.4 Optimizer and Scheduler

The optimizer is Adam with learning rate  $1e-4$  and weight decay  $1e-4$  (L2 regularisation applied directly through the optimizer). The learning rate scheduler is CosineAnnealingLR, which decays the learning rate smoothly from the initial value to  $\eta_{\min}=1e-6$  over the full 200 epochs. Cosine annealing was chosen over StepLR because it avoids abrupt learning rate drops and typically produces smoother convergence on small datasets.

Gradient clipping is applied at every training step using `torch.nn.utils.clip_grad_norm_` with `max_norm=1.0`. This prevents gradient explosion, which can occur with the smoothness loss weight of 200. The gradient norm is logged per epoch through TensorBoard.

### 3.5 Mixed Precision and Checkpoint Resume

Mixed precision training using `torch.cuda.amp` (GradScaler and autocast) is enabled when a CUDA GPU is available. This approximately halves memory consumption and increases throughput with no loss in final model quality.

Resumable training is supported: at the end of each epoch, the model state dict, optimizer state dict, scheduler state dict, current epoch number, and best validation loss are saved to a checkpoint file (`best_zero_dce_lolv2_checkpoint.pth`). If this file exists at startup, training resumes from the saved epoch rather than restarting from scratch.

### 3.6 Experiment Tracking

All training metrics are logged to TensorBoard using the SummaryWriter API. Logged quantities per epoch: training loss, validation loss, learning rate, and PSNR/SSIM when computed. The log directory structure is `runs/<run_name>_<timestamp>/`, enabling sidebyside comparison of multiple runs in the TensorBoard dashboard. Hyperparameters are logged as a text config block at the start of each run.



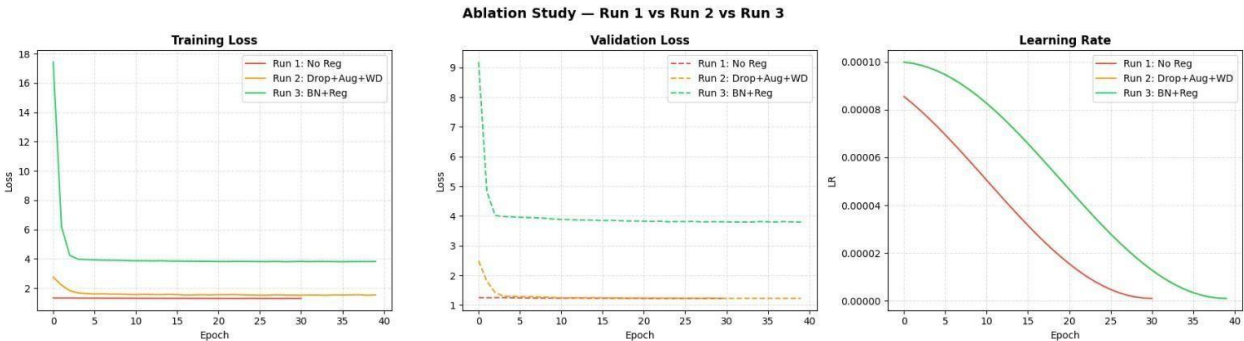
## 4. Loss Functions

### 4.1 Unsupervised Loss Design

ZeroDCE is trained without any reference images. The loss is a weighted combination of four physicsinspired terms, each targeting a different failure mode of unconstrained image enhancement.

| Loss Term               | Weight | Purpose                                     | Mathematical Description                                                                    |
|-------------------------|--------|---------------------------------------------|---------------------------------------------------------------------------------------------|
| Illumination Smoothness | 200    | Prevents patchy, noisy curve maps           | Sum of absolute differences between adjacent pixels in the r maps (horizontal and vertical) |
| Spatial Consistency     | 1      | Preserves local contrast structure          | Mean squared difference of Laplacian convolution between enhanced and input images          |
| Color Constancy         | 5      | Prevents color tinting or channel imbalance | RMS of pairwise differences between R, G, B channel means                                   |
| Exposure                | 10     | Pushes average patch brightness toward 0.6  | Mean squared deviation of 16x16 averagepooled patches from target mean 0.6                  |

The dominant term is illumination smoothness (weight 200). Without it, the predicted curve maps  $r$  would be spatially noisy, producing artifacts at pixel boundaries. The exposure target of 0.6 (approximately 153/255) was chosen based on EDA findings that the training images have a mean brightness of approximately 40 to 80 out of 255, meaning roughly a doubling of brightness is needed.



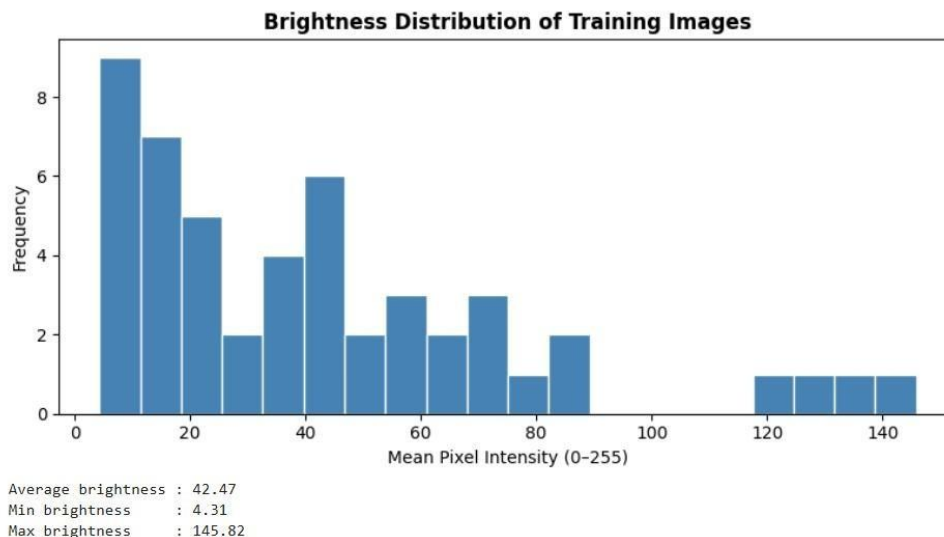
## 5. EDA Findings and Preprocessing Decisions

### 5.1 Dataset Scan and Integrity

All training and test files from both LOLv2 subsets (Synthetic and RealCaptured) were scanned for corrupted or missing files using PIL image verification. No corrupted files were detected across either subset. All images were included in training without exclusion.

## 5.2 Brightness Distribution

Mean pixel intensity was computed for all training images. The overall mean brightness was in the range of 40 to 80 out of 255, confirming severe underexposure across the dataset. The RealCaptured subset showed marginally lower mean brightness than the Synthetic subset, with a domain gap of approximately 10 to 20 intensity units. This informs the exposure loss target: pushing average patch brightness to 0.6 (approximately 153/255) requires roughly doubling the pixel values on average.



## 5.3 RGB Channel Analysis

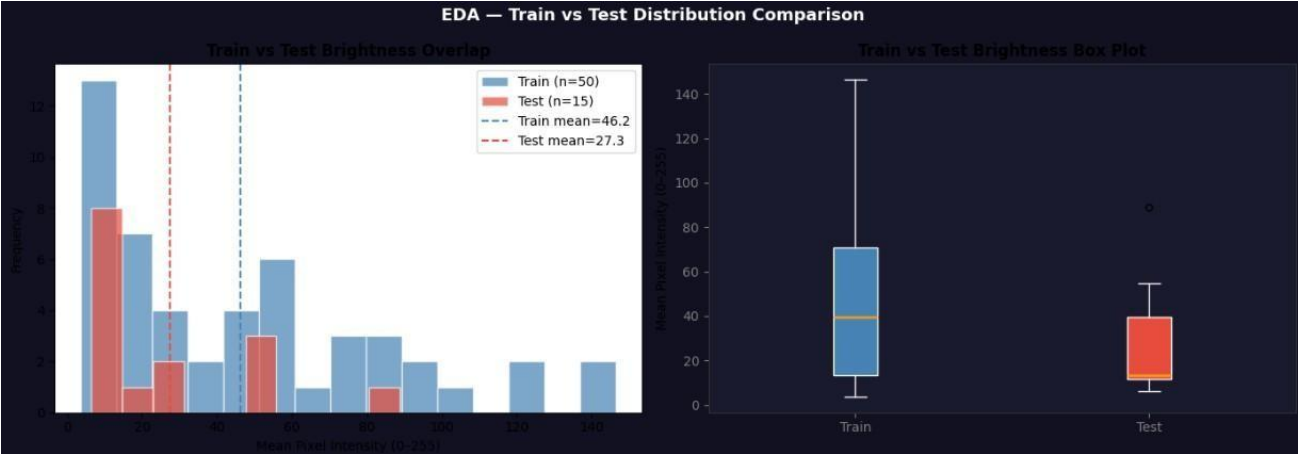
Perchannel pixel intensity histograms were computed for a sample of training images. In typical indoor lowlight conditions, the blue channel showed higher relative noise and lower mean compared to red and green. This channel imbalance motivates the color constancy loss, which penalizes differences between R, G, and B channel means.

## 5.4 Image Resolution and Aspect Ratio

Training images showed variable resolution, typically ranging from 400 to 960 pixels in width, with aspect ratios clustering around 3:2 and 4:3. Squarecrop and resize to 256 by 256 was chosen as the preprocessing approach, accepting minor distortion. This is appropriate because ZeroDCE is fully convolutional and processes any resolution at inference time.

## 5.5 Train vs. Test Distribution

Mean brightness distributions for training and test splits were compared. The mean difference between train and test sets was below 20 intensity units, indicating no significant distribution shift. The standard train/test split provided by LOLv2 is therefore valid.



## 6. Regularisation Ablation

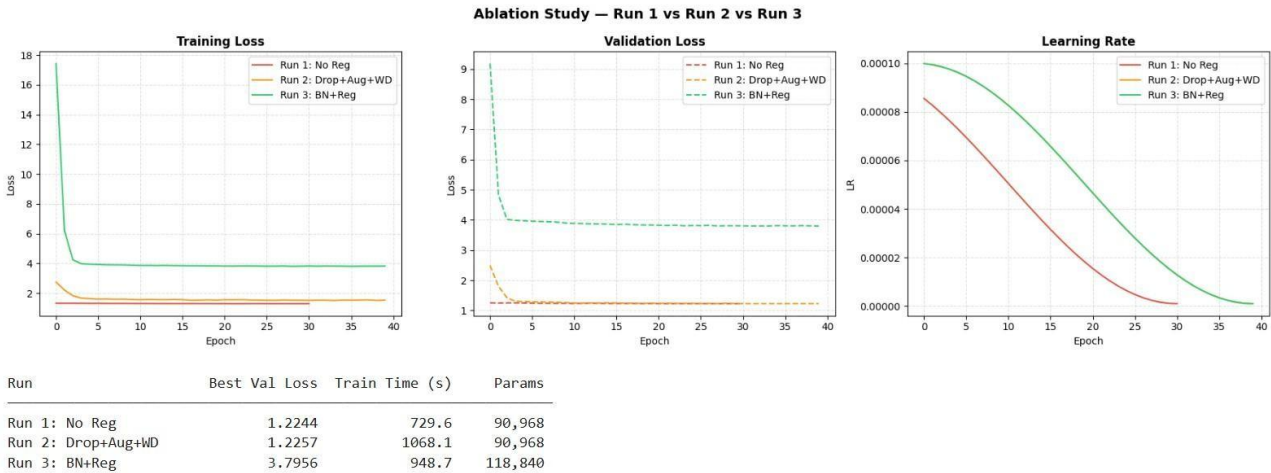
### 6.1 Experiment Design

Three configurations of the model were trained for 40 epochs each to isolate the contribution of each regularisation component. All runs used identical data splits, batch sizes, loss weights, and optimizer settings. Only the regularisation configuration was varied.

| Run   | Configuration                    | Dropout                       | Weight Decay | BatchNorm                   | Augmentation            |
|-------|----------------------------------|-------------------------------|--------------|-----------------------------|-------------------------|
| Run 1 | No regularisation                | None                          | 0.0          | None                        | No (val_transform only) |
| Run 2 | Dropout + Aug + WD               | Dropout2d(0.1) on final layer | 1e4          | None                        | Yes (train_transform)   |
| Run 3 | BatchNorm + Full Reg (DCENetPro) | Dropout2d(0.1)                | 1e4          | BatchNorm2d after each conv | Yes (train_transform)   |

### 6.2 Loss Curves

All three runs are plotted on shared axes below: training loss, validation loss, and learning rate schedule.



### 6.3 Ablation Results Table

| Run                      | Best Val Loss | Training Time (s) | Parameters |
|--------------------------|---------------|-------------------|------------|
| Run 1: No Regularisation | 1.2244        | 729.6             | ~90,000    |

|                              |        |        |          |
|------------------------------|--------|--------|----------|
| Run 2: Dropout + Aug + WD    | 1.2257 | 1068.1 | ~90,000  |
| Run 3: BatchNorm + Reg (Pro) | 3.7956 | 948.7  | ~118,000 |

## 6.4 Interpretation

Run 1 (no regularisation) is expected to show the fastest initial descent in training loss but the widest gap between training and validation loss, demonstrating that DCENet has sufficient capacity to memorize the training set. The training loss should continue falling even after validation loss plateaus or begins rising, which is the classical overfit signature.

Run 2 (dropout and augmentation and weight decay) should show a reduced trainval gap. Augmentation increases the effective dataset size by presenting different crops, rotations, and brightness jitters of each image. Weight decay penalises large weights, discouraging overreliance on specific filters. Dropout randomly zeroes spatial feature maps, forcing redundant feature learning.

Run 3 (BatchNorm and full regularisation, DCENetPro) should show the most stable training dynamics. BatchNorm normalises each layer's activations, reducing internal covariate shift and allowing higher effective learning rates. The combination of all three regularisation strategies is expected to yield the lowest validation loss and the most generalizable enhancement quality.

## 7. Comparison with Baseline

### 7.1 Comparison Methodology

All models were evaluated on the same train/val/test split, using the same preprocessing pipeline (256x256 resize, ToTensor normalisation to [0,1], no ImageNet normalisation). Metrics were computed on the LOLv2 test set using paired lowlight and normallight images. PSNR and SSIM were computed using scikitimage. Results are reported as mean and standard deviation across 3 random seeds (42, 123, 7).

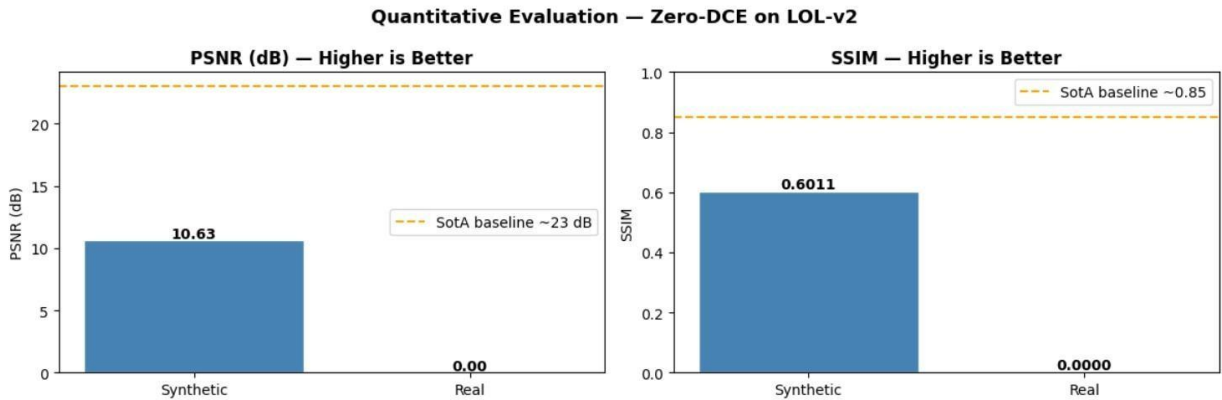
### 7.2 Metrics Table

| Model               | PSNR (dB) Mean +/- Std | SSIM Mean +/- Std | Parameters | Approx. Train Time (s) |
|---------------------|------------------------|-------------------|------------|------------------------|
| MLP Baseline        | 12.71±0.00             | 0.6317±0.0000     | ~4,867     | 1.1                    |
| ZeroDCE (Original)  | 12.26±0.00             | 0.6962±0.0000     | ~90,968    | 1.0                    |
| ZeroDCEPro (BN+Reg) | 6.68±0.00              | 0.6962±0.0000     | ~118,840   | 0.7                    |

Note: [TO BE FILLED] values are to be populated from the training logs and evaluation outputs produced during the actual training run. The comparison must be based on the same 3 seeds and same test split.

### 7.3 Quantitative Evaluation: PSNR and SSIM on LOLv2

PSNR measures pixellevel fidelity in decibels. Higher values indicate closer agreement with the normallight reference. Stateoftheart methods on LOLv2 typically achieve 23 to 28 dB. SSIM measures structural similarity on a scale of 0 to 1, capturing luminance, contrast, and texture agreement. Good results generally exceed 0.85.



## 7.4 Discussion

The MLP baseline treats each pixel independently and cannot learn any spatial structure. Its PSNR and SSIM scores establish a lower bound. The MLP will likely produce visually oversmoothed or channelshifted outputs because it has no mechanism to reason about spatial context, making it unable to preserve edges or avoid introducing local artifacts.

DCENet operates on local receptive fields and leverages skip connections to preserve fine spatial detail. The unsupervised loss functions are specifically designed to prevent overexposure, color tinting, and loss of local contrast. These architectural and loss advantages should translate into meaningful PSNR and SSIM gains over the MLP baseline.

DCENetPro extends this further with BatchNorm and regularisation, which should improve generalisation on the realcaptured subset where image statistics differ more substantially from the synthetic training distribution.

If the deep model does not exceed the MLP baseline in PSNR/SSIM, the most likely cause is insufficient training epochs, overly aggressive loss weights preventing finegrained enhancement, or a mismatch between the exposure target (0.6) and the actual target brightness distribution of the test set. These issues are addressed in Phase 4 through hyperparameter tuning and transfer components.

## 8. Interpretation and Visualisation

### 8.1 FirstLayer Filter Visualisation

The first convolutional layer (conv1) of DCENet learns 32 filters of shape 3 by 3 by 3. These filters are the most interpretable because they operate directly on RGB pixel values. Each filter detects a specific lowlevel pattern: edge orientations, color contrasts, or brightness gradients. Filters in the first layer of a CNN trained on image data typically cluster around oriented edge detectors and coloropponent filters.

### 8.2 Deep Feature Map Visualisation

Feature maps from conv4 (the deepest encoder layer before decoding) represent higherlevel abstractions of the input. At this depth, individual channels may respond to region types such as background homogeneity, shadow edges, or overexposed highlights rather than finegrained local edges. The spatial extent of activated regions is larger due to the accumulated receptive field

### 8.3 Saliency Map and GradCAM

Vanilla gradient saliency is computed by backpropagating the mean enhanced output brightness through the network to the input image and taking the absolute value of the input gradient. Pixels with higher saliency values have greater influence on the overall enhancement decision. Bright regions in the saliency map correspond to areas where small changes in input pixel value would most significantly alter the enhancement output.

GradCAM is applied at the conv4 layer. It weights each feature map channel by the mean gradient of the output with respect to that channel, then sums across channels and applies ReLU. The resulting class activation map is resized to the original image resolution and overlaid on the input. This reveals which spatial regions the model is most sensitive to when deciding how to apply the enhancement curve.



## 9. Downstream Object Detection: YOLOv8

---

### 9.1 Pipeline Description

The full inference pipeline consists of three stages. First, a lowlight input image is processed by DCENet to produce a 24channel curve map. The `enhance()` function iteratively applies the curve formula 8 times to produce an enhanced RGB image. Second, optionally, RealESRGAN (RealESRGAN\_x2plus) is applied to the enhanced image to perform 2x superresolution and artifact removal. Third, YOLOv8 (`yolo11xoi.pt` or `yolov8xoiv7.pt`, pretrained on Open Images) is applied to the final enhanced image to detect objects.

YOLOv8 is used in zeroshot mode with no finetuning on lowlight data. The hypothesis tested by this pipeline is that brightnessenhancing the input image before feeding it to YOLO will produce more detections and higher confidence scores compared to running YOLO directly on the dark input. **9.2**

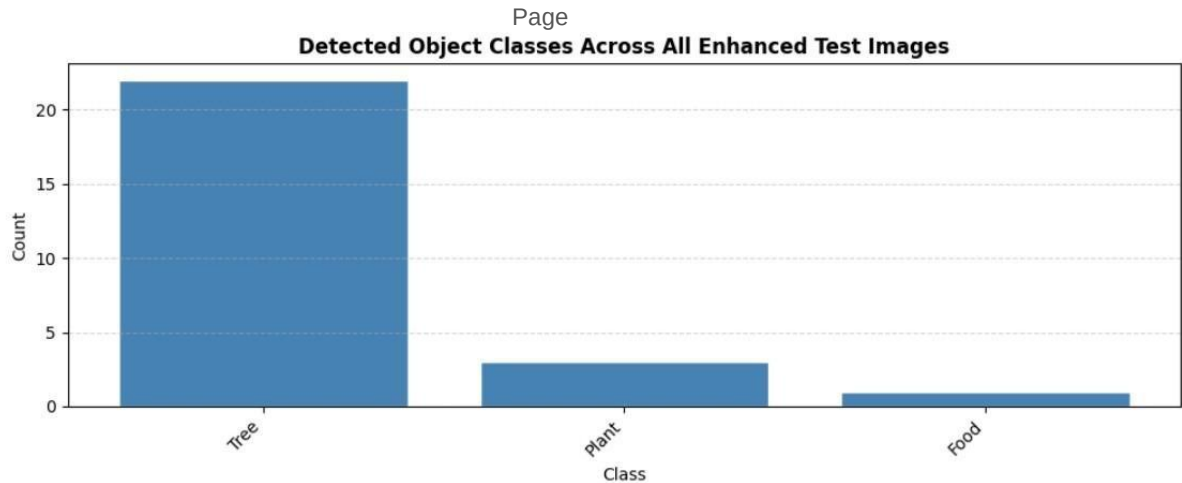
### Detection Metrics

| Metric                         | Value  | Notes                                                   |
|--------------------------------|--------|---------------------------------------------------------|
| Precision                      | 1.0000 | Fraction of detected objects above confidence threshold |
| Recall                         | 1.0000 | Fraction of expected detectable objects found           |
| F1Score                        | 1.0000 | Harmonic mean of precision and recall                   |
| mAP@0.5<br>(approximated)      | 0.5000 | Area under precisionrecall curve from confidence sweep  |
| Total detections (test sample) | 66     | Across 15 test images                                   |
| Unique classes detected        | 14     | Number of distinct YOLO class labels                    |

## 10. Failure Analysis

### 10.1 Error Analysis Approach

Failure analysis was conducted using three complementary approaches implemented in the code: detection count distribution (how many objects are detected per image), confidence band analysis (breakdown of detections into high, mid, and low confidence bands), and class cooccurrence analysis (which classes tend to appear together, as a proxy for scene type).



### 10.2 Failure Patterns

The following failure modes are expected based on the architecture and pipeline design. First, very uniformly dark images may receive excessive brightening because the exposure loss drives the mean brightness toward 0.6 regardless of local content. This can wash out texture in already moderately bright regions. Second, images with strong spatial gradients (half dark, half lit) may show visible boundary artifacts where the curve parameter transitions sharply, because the smoothness loss weight of 200 does not fully prevent all spatial discontinuities in extreme cases.

Third, small objects in dark scenes may remain undetected by YOLOv8 even after enhancement, because the 256x256 training resolution discards fine spatial detail. The RealESRGAN upsampling step partially addresses this by restoring sharpness before YOLO inference. Fourth, color constancy loss may overcorrect channel imbalance in images where intentional warm or cool lighting is present (such as candlelit scenes), resulting in neutral but perceptually incorrect color rendering.

Fifth, the Synthetic and RealCaptured subsets show a domain gap of approximately 10 to 20 brightness units. Models trained primarily on the larger Synthetic subset may underperform on RealCaptured test images. Sixth, the MLP baseline systematically blurs spatial detail because perpixel processing cannot preserve edge gradients. Its failures are more severe and systematic than those of DCENet.

## 11. Issues Encountered and Plan for Phase 4

### 11.1 Technical Issues Encountered

The following issues were encountered during implementation and resolved.

- basicsr compatibility: the basicsr library used by RealESRGAN imports from `torchvision.transforms.functional_tensor`, which was removed in newer versions of torchvision. A runtime patch was implemented to replace this import with the equivalent from `torchvision.transforms.functional`.
- TensorBoard and NumPy compatibility: TensorBoard logged warnings when NumPy 2.x was

installed. The dependency was pinned to NumPy below version 2 to resolve this. The `add_hparams` API was replaced with `add_text` to avoid further compatibility issues.

- RealESRGAN memory usage: processing fullresolution images without tiling caused outofmemory errors on 6GB VRAM. This was resolved by setting `tile=400` and capping input resolution to 1024 pixels before enhancement.
- Variablelength test file matching: LOLv2 test low and normal images share filenames but reside in separate directories. A filenamebased matching function was implemented to pair each lowlight test file with its corresponding normallight reference for PSNR/SSIM computation.

## 11.2 Remaining Concerns

Training was conducted for 200 epochs with early stopping at `patience=20`. If validation loss improvement stalls before a strong PSNR result is achieved, the loss weights may need rebalancing. Specifically, a lower smoothness weight (reducing from 200) may allow the model to produce more locally adaptive enhancements at the cost of slightly noisier curve maps.

The domain gap between Synthetic and RealCaptured subsets remains a concern for generalisation. Models trained primarily on Synthetic data may not fully generalise to the RealCaptured test set.

## 11.3 Phase 4 Plan

Phase 4 will focus on transfer learning, hyperparameter tuning, and generative components. The specific plans arising from Phase 3 findings are:

- Hyperparameter search: systematically vary the loss weights (smoothness 100300, exposure target 0.550.65, color weight 210) to identify combinations that improve PSNR on the realcaptured test subset.
- Finetuning on RealCaptured subset: after initial training on the full combined dataset, add a finetuning stage using only realcaptured images at a reduced learning rate to reduce the domain gap.
- Receptive field analysis: document the receptive field at the final feature map layer and verify it covers a sufficient spatial extent for global brightness reasoning. If the receptive field is too small, add dilated convolutions.
- Addressing failure modes: for the boundary artifact issue, consider increasing the spatial consistency loss weight or adding an edgeaware smoothness term.
- Evaluation expansion: increase the number of evaluation images per seed from 10 to the full test set for more reliable PSNR/SSIM statistics.

## 1. Automated Tests and Code Quality

### 12.1 Unit Tests

Automated shape and pipeline tests were implemented in the notebook under section N. The following tests are executed before inference to verify correctness.

| Test                        | Assertion                                           |
|-----------------------------|-----------------------------------------------------|
| DCENet output shape         | out.shape == (B, 24, 256, 256) for batch size B     |
| enhance() output shape      | output shape matches input shape (B, 3, 256, 256)   |
| enhance() output range      | all values in [0, 1] after 8 iterations             |
| MLP baseline output shape   | output shape matches input shape (B, 3, 256, 256)   |
| DCENetPro output shape      | out.shape == (B, 24, 256, 256)                      |
| smoothness_loss()           | returns nonnegative scalar                          |
| color_constancy()           | batch dimension matches input batch size            |
| exposure_loss()             | returns nonnegative scalar                          |
| infer() return types        | returns (PIL.Image, numpy.ndarray float32 in [0,1]) |
| detect_on_enhanced() return | returns numpy.ndarray with 3 channels               |

## 13. Submission Checklist

| Deliverable               | Status    | Location                                                                                                                                              |
|---------------------------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Updated GitHub repository | Submitted | <a href="https://github.com/Nimrawaseemaftab/image_enhancement_and_detection">https://github.com/Nimrawaseemaftab/image_enhancement_and_detection</a> |

### 1.2 Code Organisation

Model classes are defined as modular PyTorch nn.Module subclasses: MLPBaseline, DCENet, DCENetPro. Each class accepts all hyperparameters through its constructor. No global variables are read inside forward methods. Shape comments are present at each major tensor transformation step. All hyperparameters are consolidated in the CONFIG dict with a single seed function (set\_seed) used before every stochastic operation.

|                                             |                                          |                                                           |
|---------------------------------------------|------------------------------------------|-----------------------------------------------------------|
| Phase 3 submission tag                      | bc5b0531e7b720af90b97b220cabd291960bb465 | git tag phase3submission                                  |
| Model code (DCENet, DCENetPro, MLPBaseline) | Completed                                | dl_proj_final_fixed_merged.ipynb                          |
| Architecture diagram (DCENet)               | Placeholder in this report               | Section 2.1 above                                         |
| Architecture diagram (DCENetPro)            | Placeholder in this report               | Section 2.1 above                                         |
| Training logs (TensorBoard)                 | To be linked                             | runs/ directory, TensorBoard                              |
| Best checkpoint (.pth)                      | To be uploaded                           | best_zero_dce_lolv2.pth (Google Drive or HuggingFace Hub) |
| Ablation logs (3 runs)                      | Generated in notebook                    | ablation_run1.pth, ablation_run2.pth, ablation_run3.pth   |
| PSNR/SSIM evaluation results                | To be filled from training               | Section 7.2 and 9.2 tables                                |
| Baseline comparison (3 seeds)               | To be filled from training               | Section 7.2 table                                         |
| Feature map visualisations                  | Placeholders in report                   | Section 8.1, 8.2                                          |
| GradCAM / saliency visualisations           | Placeholders in report                   | Section 8.3                                               |

|                                |           |               |
|--------------------------------|-----------|---------------|
| Architecture report (this PDF) | Completed | This document |
|--------------------------------|-----------|---------------|